Relatório 2 - Prática: Linguagem de Programação Python (I)

Matheus Beuren Carvalho

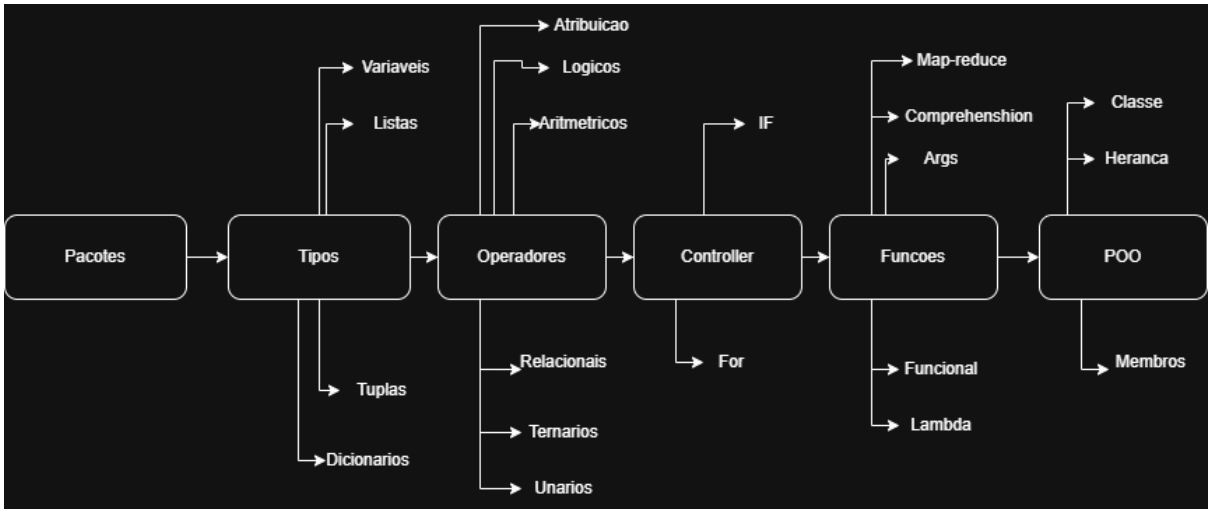
1. Introdução

Conforme a tarefa descrita no card, foi pedido a visualização e pratica dos conceitos dos dois vídeos apresentados e a compreensão dos conceitos. Os vídeos em questão, referem-se ao aprendizado da linguagem python, amplamente usada para projetos futuros como em ML, DL e LLM. Deste modo, foi realizado este relatório com a intenção de mostrar os conhecimentos aprendidos nos vídeos.

2. Desenvolvimento

No contexto de aprendizado, o curso disponibilizado teve como estrutura final, separada em conceitos, como Pacotes, Tipos, Operadores, Controladores, Funções e POO, como apresentado na **Figura 1.**

Figura 1 - Fluxograma dos modulos



A linguagem Python é a linguagem mais utilizada e diversificada atualmente. Assistindo ao conteúdo designado nos vídeos, mesmo com o conhecimento prévio da linguagem, foram abordados conhecimentos básicos e avançados.

OBS: Utilizei python 3 via Anaconda, pode-se notar no código que utilizei pequenos comandos para surtir do mesmo efeito do curso

Começo descrevendo os tipos e variáveis. Consistem na mesma simbologia que em outras linguagens como o C; variáveis são como caixas que guardam informações. Em Python, como é tipada, reconhece qualquer atribuição, diferente de outras linguagens em que deve sempre colocar-se o tipo da variável. Os tipos de variáveis seguem o mesmo padrão do que para outras linguagens, como int, float, string, etc.

Avançando ao conteúdo, chegamos à parte de Listas, Tuplas, Conjuntos e Dicionários. As listas são uma sequência de argumentos sendo mutáveis, tendo como principal função adicional o append(), que adiciona à lista. Tuplas, por si, são como listas, mas são imutáveis, fazendo assim, após a criação, impossibilitando a modificação e nem atribuição a tal. Os conjuntos são uma coleção mutável de valores distintos. Por fim, dicionários são feitos com palavra-chave valor, se assimilando à struct em C, sendo uma das mais utilizadas em Python.

Operadores são um conjunto de conceitos mais utilizados em Python; os aritméticos vêm das operações básicas de matemática como: adição, subtração, multiplicação, divisão e resto da divisão. Os lógicos são destinados à comparação, tendo o E equivalente a AND e OU equivalente ao OR; comparações lógicas usadas para ver se duas premissas são ambas verdadeiras no caso de E retorna True, sendo qualquer uma false já retorna False; o OR basta somente uma ser verdadeira. Já os ternários avaliam para um de dois valores, dependendo se a condição é verdadeira ou falsa.

Chegando às estruturas de controle, temos o if, que compara uma afirmativa com alguma condição; se for verdadeiro, então entra no bloco if, onde são executados comandos. Diante de dois loops, for e o while, a diferença é que o for itera sobre uma sequencia (range, lista), já o while faz o loop enquanto uma condição for verdadeira.

Em uma função, é definido algo a ser feito dentro dela contendo os comandos que, ao ser chamada, é executada, podendo receber como parâmetro variáveis, números, etc. Aí surgem os *args e os **kwargs, que mostraram funções que podem receber diversos parâmetros de uma única vez. Também foi possível analisar na prática como as funções são tratadas como objetos em Python, podendo ser usadas em parâmetros de outras funções e ser atribuídas a variáveis.

Sobre a programação funcional, foram abordados conceitos como map, reduce e lambda. O conceito map altera todos os elementos de uma lista ou dicionário; o filter retorna somente o que passou pelas condições propostas; reduce é usado para transformar listas ou sequências em um único resultado; por fim, lambda é uma função específica para um caso, criada de uma forma rápida, sendo utilizada em ML para manipulação com arrays onde só precisa fazer algo especifico com ele.

O List Comprehension, por sua vez, coloca ifs e laços de repetição em uma única linha, poupando espaço e otimizando o código, sendo eficiente para quem já tem experiência.

A parte do encapsulamento demonstrou a importância de controlar o acesso aos atributos de uma classe. A utilização de atributos privados, junto com os métodos property e setter, permitiu compreender como os dados podem ser protegidos e modificados de maneira controlada, evitando alterações não autorizadas e aumentando a segurança das informações. Como exemplo, cito oque está no código de prática em anexo, onde existe um personagem com atributo __vida e um setter que o impede de ser negativo.

Tabela 1 - Diferenças de sintaxe Python x C

Comando	**Python**	**C**
Mostrar na tela	print(‘ ’)	printf(“ ”);
Ler entrada usuario	input(‘Digite um numero:’)	scanf(“%d”,&num);
Função	def	tipo de retorno ou void
Loop FOR	for i in range(1,10)	for(int i = 1;i<10;i++)

3. Conclusão

Ao longo deste relatório, foi apresentado o conteúdo aprendido nos vídeos passados no card sobre a linguagem Python. Mesmo com o conhecimento prévio da linguagem, os vídeos abordaram desde os fundamentos, como tipos e variáveis, estruturas de dados (Listas, Tuplas, Conjuntos e Dicionários), operadores e estruturas de controle (if, for, while), até conceitos mais avançados. Destaco a versatilidade das funções, incluindo o uso de *args e **kwargs, e a aplicação de programação funcional com map, reduce e lambda, além da otimização proporcionada pelo List Comprehension.Os conceitos aprendidos serão usados futuramente como citados no Bootcamp. Concluo este relatório, que aborda os principais conhecimentos adquiridos, reforçando que este aprendizado é essencial para aplicações de ciência de dados e ML.